Built-in Data Structures

Lecture 3 · Learning Python at the University of Glasgow

Khiem Nguyen

9 October 2026

Contents

1 Overview: collections of values **2**

2 Lists **2**
2.1 Creating lists . . . 2
2.2 Lists, `for` loops and `range()` . . . 3
2.3 Accessing elements: indexing and slicing . . . 4
2.4 Modifying lists . . . 6
2.5 List methods . . . 7
2.6 Copying a list versus creating an alias . . . 8
2.7 Lists of lists . . . 10

3 Tuples **14**
3.1 Creating tuples . . . 14
3.2 Accessing elements . . . 14
3.3 Tuples are immutable . . . 15
3.4 Packing and unpacking . . . 16

4 Dictionaries **17**
4.1 Creating dictionaries . . . 17
4.2 Accessing, adding and deleting elements . . . 17
4.3 Looping over a dictionary . . . 19

5 Sets **19**

6 Useful looping techniques **20**

Summary **21**

Exercises **21**
Exercise 1: List methods . . . 21
Exercise 2: Printing a 2D list in style . . . 21
Exercise 3: A dictionary of material densities . . . 22
Exercise 4: Lists versus NumPy for matrix multiplication . . . 22

Further reading **23**

Learning objectives

By the end of this lecture, you should be able to:

- create lists, tuples, dictionaries and sets, and choose the right one for a task;
- access elements by indexing and slicing, including negative indices and steps;
- modify lists with assignments and list methods;
- explain the difference between copying a list and creating an alias of it;
- build and process nested lists (lists of lists);
- pack and unpack tuples, and loop over dictionaries;
- use the looping helpers `enumerate()` and `zip()`.

1 Overview: collections of values

So far, each variable has held a single value. Python also provides several **compound** data types, which group many values together. In this lecture, we study the three most widely used ones — the **list**, the **tuple** and the **dictionary** — and take a brief look at the **set**.

	List	Tuple	Dictionary	Set
Example	`[1, 'two', 3]`	`(1, 'two', 3)`	`{'one': 1, 'two': 2}`	`{1, 2, 3}`
Access to the elements	by position (index)	by position (index)	by key	no direct access
Mutable (can change)?	yes	**no**	yes	yes
Duplicates	allowed	allowed	keys must be unique	not allowed
Typical use	a collection that changes	a fixed group of values	look-up by name	unique items, membership tests

All of them are **objects** equipped with useful *methods*. We will study objects properly in Lecture 5 on object-oriented programming.

2 Lists

2.1 Creating lists

We have already met lists in Lecture 2. A list is written as a sequence of comma-separated values (items) between square brackets:

```
[item_1, item_2, item_3, ...]
```

- The items need not have the same type: a list can contain integers, floating-point numbers, strings — or any other kind of data, including other lists. We say that a list can hold **heterogeneous** data, although in practice the items usually all have the same type.
- A list is **mutable**: we can change its items after it has been created.
- The built-in function `len()` returns the number of items in a list.

```
number_list = [5.5, 6.5, 7.5, 8.5, 9.5, 10.5]     # a list of numbers
string_list = ['We', 'are', 'the', 'champions.']  # a list of strings
# mixed types are allowed
mixed_type_list = [5.5, 'We', 6.5, 'are', 'the', 'champions']
empty_list = []                                   # an empty list

print(len(number_list), len(string_list), len(empty_list))
```

```
6 4 0
```

2.2 Lists, `for` loops and `range()`

A list is frequently used in a `for` loop. In the statement

```
for element in looping_list:
    print(element)
```

the variable `element` takes the value of each item of `looping_list` in turn, from the first item to the last.

```
for x in string_list:
    print(x, end=" ")
```

```
We are the champions.
```

A list of integers can be created with `list(range(...))`, where the arguments of `range()` work exactly as in Lecture 2. Note that `range()` on its own does not return a list but a *range object*, which produces the numbers one by one when needed; `list()` collects all of them into a list.

```
range_list = list(range(5, 10, 1))   # the numbers from 5 to 9
print("range_list =", range_list)
```

```
odd_list = list(range(1, 10, 2))     # the odd numbers from 1 to 9
print("odd_list =", odd_list)

even_list = list(range(2, 10, 2))    # the even numbers from 2 to 8
print("even_list =", even_list)
```

```
range_list = [5, 6, 7, 8, 9]
odd_list = [1, 3, 5, 7, 9]
even_list = [2, 4, 6, 8]
```

`range()` only works with integers. If the start, the stop or the step is not an integer, Python raises an error:

```
list(range(1.2, 5, 1))   # TypeError: range() needs integers
```

```
TypeError: 'float' object cannot be interpreted as an integer
---------------------------------------------------------------------------
TypeError                                 Traceback (most recent call last)
Cell In[41], line 1
----> 1 list(range(1.2, 5, 1))   # TypeError: range() needs integers
TypeError: 'float' object cannot be interpreted as an integer
```

> **Non-integer steps**
>
> For evenly spaced floating-point values, use NumPy instead: `np.arange(1.2, 5, 0.5)` or `np.linspace(0, 1, 11)`.

2.3 Accessing elements: indexing and slicing

Indexing. To access a single element of a list, we use its position, or **index**, in square brackets. If `my_list` has 10 elements, `my_list[0]` is the first element, `my_list[1]` is the second one, and so on, up to `my_list[9]`, the last one. Remember that counting starts from **zero** — unlike in MATLAB, where it starts from one.

Negative indices count from the end: `my_list[-1]` is the last element, `my_list[-2]` the second-to-last element, and so on. So, for a list of 10 elements, `my_list[9]` and `my_list[-1]` are the same element, as are `my_list[8]` and `my_list[-2]`. Notice that

$$9 - 10 = -1, \qquad 8 - 10 = -2.$$

In general, if `my_list` has N elements and $0 \le m < N$, then `my_list[m]` and `my_list[m - N]` refer to the same element.

Slicing. While indexing returns a single element, **slicing** returns a sub-list of the original list:

```
my_list[start:stop]  # the elements with indices start, start + 1, ...,
 ↪  stop - 1
# every step-th element from start up to (but excluding) stop
my_list[start:stop:step]
```

As with `range()`, the `stop` index is **excluded**. For example, `my_list[0:2]` returns the elements with indices 0 and 1 — two elements, not three. In general, `my_list[n:m]` with $n < m$ returns $m - n$ elements. If `start` is omitted, the slice begins at the start of the list; if `stop` is omitted, it runs to the end.

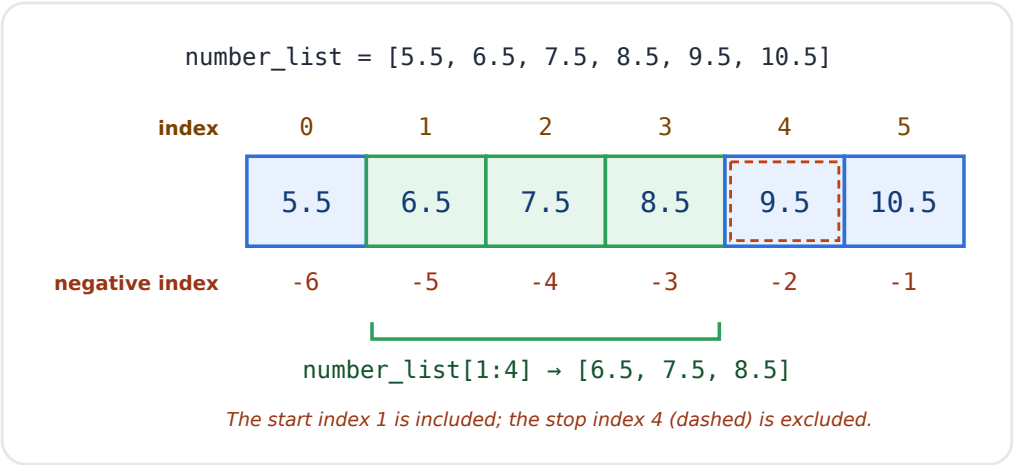


Figure 1: Indexing and slicing the list `number_list`. Each element can be reached with a positive or a negative index. The slice `number_list[1:4]` contains the elements with indices 1, 2 and 3.

```
number_list = [5.5, 6.5, 7.5, 8.5, 9.5, 10.5]
print("number_list =", number_list)
print(number_list[0])                    # the first element
print(number_list[-1])                   # the last element
print(number_list[len(number_list) - 1])   # also the last element
```

```
number_list = [5.5, 6.5, 7.5, 8.5, 9.5, 10.5]
5.5
10.5
10.5
```

```
print(number_list[1:4])   # indices 1, 2, 3 (4 is excluded), as in the
 ↪  figure
print(number_list[0:4])   # the first four elements: indices 0, 1, 2, 3
```

```
print(number_list[:3])    # from the beginning up to index 2
print(number_list[2:])    # from index 2 to the end
print(number_list[::2])   # every second element
print(number_list[::-1])  # the whole list, reversed
# slicing beyond the end is fine: the slice just stops at the end
print(number_list[:7])
```

```
[6.5, 7.5, 8.5]
[5.5, 6.5, 7.5, 8.5]
[5.5, 6.5, 7.5]
[7.5, 8.5, 9.5, 10.5]
[5.5, 7.5, 9.5]
[10.5, 9.5, 8.5, 7.5, 6.5, 5.5]
[5.5, 6.5, 7.5, 8.5, 9.5, 10.5]
```

Indexing beyond the end, however, is an error:

```
number_list[10]    # IndexError: list index out of range
```

```
IndexError: list index out of range
```

```
---------------------------------------------------------------------------
IndexError                                Traceback (most recent call last)
Cell In[44], line 1
----> 1 number_list[10]    # IndexError: list index out of range
IndexError: list index out of range
```

Slicing works in exactly the same way for a list of mixed types:

```
print("mixed_type_list =", mixed_type_list)
print(mixed_type_list[1:5])
```

```
mixed_type_list = [5.5, 'We', 6.5, 'are', 'the', 'champions']
['We', 6.5, 'are', 'the']
```

2.4 Modifying lists

Because lists are mutable, we can assign new values to individual elements or even to whole slices. The membership operators `in` and `not in` from Lecture 1 also work with lists.

```
numbers = [1, 2, 3, 4, 5]
numbers[0] = 100            # replace one element
numbers[1:3] = [20, 30]     # replace a slice
```

```
print(numbers)
print(3 in numbers, 30 in numbers)
```

```
[100, 20, 30, 4, 5]
False True
```

2.5 List methods

A list is an object with many **methods**. To call a method on a list, write

```
my_list.method_name(arg1, arg2, ...)
```

There are far too many methods to remember them all — and you should not try. Instead, use the **auto-complete** feature of your IDE (type `my_list.` and press **Tab**), and read the documentation of a particular method with `help(list.append)` (or, in Jupyter, `list.append?`). The most useful ones are:

Method	What it does
`append(x)`	adds `x` at the end of the list
`extend(iterable)`	adds all the items of `iterable` at the end of the list
`insert(i, x)`	inserts `x` before position `i`
`remove(x)`	removes the first item equal to `x`
`pop(i)`	removes and returns the item at position `i` (the last one by default)
`sort()`	sorts the list in place
`reverse()`	reverses the list in place
`index(x)`	returns the position of the first item equal to `x`
`count(x)`	returns the number of items equal to `x`
`copy()`	returns a (shallow) copy of the list — see the next section

The built-in functions `len()`, `max()`, `min()`, `sum()` and `sorted()` also work with lists.

```
x = []
x.append(1)
print(x, end="\t\t")
print("max =", max(x))
for j in range(4):              # j = 0, 1, 2, 3
    x.append(j)
    print(x, end="\t\t")
    print("max =", max(x))     # change max to min to obtain the smallest
     ↪  element
```

```
[1]     max = 1
[1, 0]      max = 1
[1, 0, 1]       max = 1
[1, 0, 1, 2]        max = 2
[1, 0, 1, 2, 3]     max = 3
```

```
fruits = ["apple", "banana"]
fruits.extend(["cherry", "date"])
fruits.insert(1, "avocado")
print(fruits)
last = fruits.pop()
print("popped:", last, "->", fruits)
fruits.remove("banana")
fruits.sort(reverse=True)
print(fruits)
print(len(fruits), sorted([3, 1, 2]), sum([3, 1, 2]))
```

```
['apple', 'avocado', 'banana', 'cherry', 'date']
popped: date -> ['apple', 'avocado', 'banana', 'cherry']
['cherry', 'avocado', 'apple']
3 [1, 2, 3] 6
```

2.6 Copying a list versus creating an alias

An assignment such as `list_alias = number_list` does **not** copy the list. It simply binds a second name to the same list object (recall the figure on names and objects in Lecture 1). Any change made through one name is therefore visible through the other. To obtain an independent copy, use `number_list.copy()` or the slice `number_list[:]`.

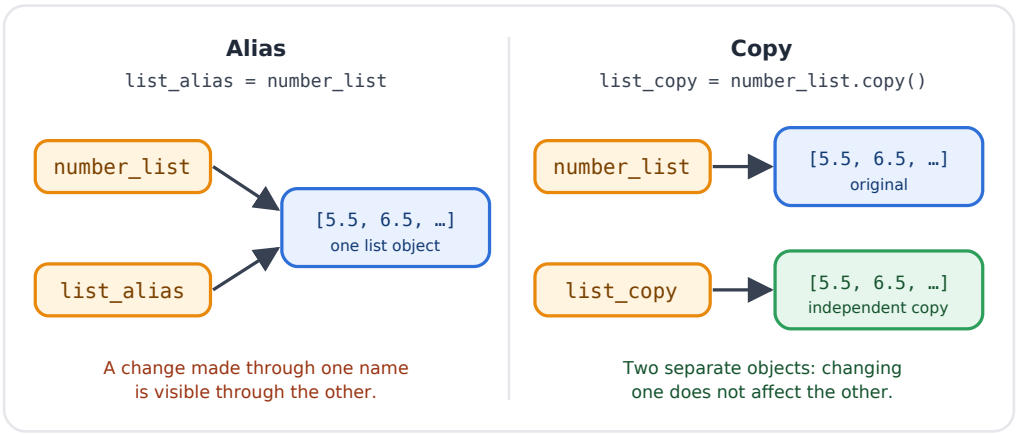


Figure 2: An alias is a second name for the same list object; a copy is a new, independent list object.

```
number_list = [5.5, 6.5, 7.5, 8.5, 9.5, 10.5]   # create number_list again

list_copy_1 = number_list[:]          # a copy of number_list
list_copy_2 = number_list.copy()      # another copy of number_list
list_copy_1[0] = 200                  # this does not change number_list ...
list_copy_1[1] = 400
print("list_copy_1 =", list_copy_1)
print("list_copy_2 =", list_copy_2)  # ... and it does not change
 ↪  list_copy_2 either
print("number_list =", number_list)
```

```
list_copy_1 = [200, 400, 7.5, 8.5, 9.5, 10.5]
list_copy_2 = [5.5, 6.5, 7.5, 8.5, 9.5, 10.5]
number_list = [5.5, 6.5, 7.5, 8.5, 9.5, 10.5]
```

```
# The three lists are three different objects, with three different IDs.
print("id(number_list) =", id(number_list))
print("id(list_copy_1) =", id(list_copy_1))
print("id(list_copy_2) =", id(list_copy_2))
print(number_list is list_copy_1)
```

```
id(number_list) = 123844218281280
id(list_copy_1) = 123844218280640
id(list_copy_2) = 123844218274432
False
```

```
number_list = [5.5, 6.5, 7.5, 8.5, 9.5, 10.5]
list_alias = number_list  # list_alias and number_list refer to the same
 ↪  object

print("Before changing list_alias:")
print("number_list =", number_list)

list_alias[0] = 0                     # this changes number_list[0] as well
list_alias[1] = 1                     # this changes number_list[1] as well

print("After changing list_alias:")
print("number_list =", number_list)
```

```
Before changing list_alias:
number_list = [5.5, 6.5, 7.5, 8.5, 9.5, 10.5]
After changing list_alias:
number_list = [0, 1, 7.5, 8.5, 9.5, 10.5]
```

```
# The two names refer to the same object, so they have the same ID.
print("id(number_list) =", id(number_list))
print("id(list_alias)  =", id(list_alias))
print(number_list is list_alias)
```

```
id(number_list) = 123844218481728
id(list_alias)  = 123844218481728
True
```

2.7 Lists of lists

Since a list can contain data of any type, a list can also contain other lists, which we call **sub-lists**. A simple example is a matrix of numbers:

```
matrix = [[1, 2, 3, 4], [5, 6, 7, 8], [9, 10, 11, 12]]
```

To access the elements of `matrix`, we again use indexing and slicing. `matrix[0]` is the first element of `matrix`, which turns out to be the sub-list `[1, 2, 3, 4]`; likewise, `matrix[1]` is `[5, 6, 7, 8]`, and `matrix[2]` is `[9, 10, 11, 12]`. We can then index into a sub-list: `matrix[0][0]`, `matrix[0][1]`, `matrix[0][2]` and `matrix[0][3]` return `1`, `2`, `3` and `4`, respectively. In general, `matrix[i][j]` is the element in row `i` and column `j`. Of course, the sub-lists can also contain strings:

```
string_grid = [['.', '.', '.'], ['.', '.', '.'], ['.', '.', '.']]
```

```
# Let us create the matrix with 3 rows and 4 columns described above.
# We start with an empty list, then append three sub-lists, one for each
 ↪  row.
matrix = []
for i in range(3):
    sublist = []                    # start each row as an empty list
    for j in range(1, 5):           # j runs from 1 to 4 (5 is excluded)
        sublist.append(4*i + j)     # append each number to the row
    matrix.append(sublist)          # append the complete row to the matrix

print(matrix)
print(matrix[1][2])                 # row 1, column 2
```

```
[[1, 2, 3, 4], [5, 6, 7, 8], [9, 10, 11, 12]]
7
```

```
# A list of lists of strings
my_list = []
for i in range(3):
    if i % 2 == 0:
        my_list.append(['o', 'x', 'o', 'x'])
    else:
        my_list.append(['x', 'o', 'x', 'o'])

print(my_list)
```

```
[['o', 'x', 'o', 'x'], ['x', 'o', 'x', 'o'], ['o', 'x', 'o', 'x']]
```

```
# We can print the list of lists neatly by visiting every element of every
 ↪ sub-list.
n = len(my_list)              # the number of sub-lists
for i in range(n):
    sublist = my_list[i]
    m = len(sublist)          # the number of elements in the sub-list
 ↪ my_list[i]
    for j in range(m):
        print(sublist[j], end="")
    print()                   # start a new line after printing a complete
     ↪ sub-list
```

```
oxox
xoxo
oxox
```

```
# The same, more compactly: refer to the sub-list as my_list[i]
# and to its j-th element as my_list[i][j].
for i in range(len(my_list)):
    for j in range(len(my_list[i])):
        print(my_list[i][j], end="")
    print()
```

```
oxox
xoxo
oxox
```

```
# Swap every 'x' for an 'o' and vice versa.
for i in range(len(my_list)):
```

```
    for j in range(len(my_list[i])):
        if my_list[i][j] == 'x':
            my_list[i][j] = 'o'
        else:
            my_list[i][j] = 'x'

# We are repeating the printing code from above. In Lecture 4, we will
 ↪ learn
# how to package it in a function so that we do not have to copy and paste
 ↪ it.
for i in range(len(my_list)):
    for j in range(len(my_list[i])):
        print(my_list[i][j], end="")
    print()
```

```
xoxo
oxox
xoxo
```

A more complicated example. We can even build a list of lists of lists, i.e. a three-level list. If its data are numbers, we can picture it as a 3D block of numbers.

```
my_numbers = []
for i in range(2):
    sublist = []
    for j in range(3):
        subsublist = []
        for k in range(4):
            # think about it: which numbers does this give?
            subsublist.append(3*4*i + 4*j + k)
        sublist.append(subsublist)  # this belongs to the j-loop
    my_numbers.append(sublist)      # this belongs to the i-loop

# my_numbers contains two matrices, each with 3 rows and 4 columns.
print("my_numbers[0] =\n", my_numbers[0])
print("my_numbers[1] =\n", my_numbers[1])
```

```
my_numbers[0] =
 [[0, 1, 2, 3], [4, 5, 6, 7], [8, 9, 10, 11]]
my_numbers[1] =
 [[12, 13, 14, 15], [16, 17, 18, 19], [20, 21, 22, 23]]
```

```
# Print the two matrices neatly
for i in range(len(my_numbers)):
    for j in range(len(my_numbers[i])):
        for k in range(len(my_numbers[i][j])):
            print("{0:<3}".format(my_numbers[i][j][k]), end="")
        print()                  # a new line after each row (j-loop)
    print("-----------")         # a separator after each matrix (i-loop)
```

```
0  1  2  3
4  5  6  7
8  9  10 11
-----------
12 13 14 15
16 17 18 19
20 21 22 23
-----------
```

⚠ Copying a list of lists

`copy()` and `[:]` make a **shallow** copy: they create a new outer list, but the sub-lists inside it are *shared* with the original. Changing a sub-list through the copy therefore changes the original too, as the next cell shows. To copy a nested list completely, use `deepcopy()` from the `copy` module.

```
grid = [[0, 0], [0, 0]]
grid_copy = grid.copy()       # copies the OUTER list only
grid_copy[0][0] = 99          # modifies an inner list that both share!
print("grid      =", grid)
print("grid_copy =", grid_copy)

import copy
grid = [[0, 0], [0, 0]]
grid_deep = copy.deepcopy(grid)   # copies everything, at every level
grid_deep[0][0] = 99
print("grid      =", grid)
print("grid_deep =", grid_deep)
```

```
grid      = [[99, 0], [0, 0]]
grid_copy = [[99, 0], [0, 0]]
grid      = [[0, 0], [0, 0]]
grid_deep = [[99, 0], [0, 0]]
```

3 Tuples

3.1 Creating tuples

A **tuple** consists of a number of values separated by commas, usually enclosed in round brackets (parentheses) instead of square brackets:

```
t = (value_1, value_2, ..., value_N)
```

The brackets may even be left out:

```
t = value_1, value_2, ..., value_N
```

Tuples often contain a heterogeneous sequence of elements that are accessed by **unpacking** (see below).

```
# a tuple containing a tuple and two integers
t1 = ((3, 4), 1, 2)
t2 = (list(range(2, 11, 2)), list(range(9, 0, -2)))  # a tuple containing
 ↪  two lists
empty_tuple = ()
# a tuple with ONE element needs a trailing comma ...
single = (5,)
# ... otherwise it is just the integer 5 in brackets
not_a_tuple = (5)

print("t1 =", t1)
print("t2 =", t2)
print("empty_tuple =", empty_tuple)
print(type(single), type(not_a_tuple))
```

```
t1 = ((3, 4), 1, 2)
t2 = ([2, 4, 6, 8, 10], [9, 7, 5, 3, 1])
empty_tuple = ()
<class 'tuple'> <class 'int'>
```

3.2 Accessing elements

Elements of a tuple are accessed exactly like those of a list, by indexing and slicing, and a tuple can be used in a `for` loop in the same way as a list.

```
print("t1[0] =", t1[0])          # the first element of t1
print("t2[1] =", t2[1])          # the second element of t2
print("t2[1][0] =", t2[1][0])    # the first element of the second element
 ↪ of t2
for element in t1:
    print(element)
```

```
t1[0] = (3, 4)
t2[1] = [9, 7, 5, 3, 1]
t2[1][0] = 9
(3, 4)
1
2
```

3.3 Tuples are immutable

The key difference from a list is that a tuple is **immutable**: once created, its elements cannot be changed. If `t = (1, 2, 3)`, the statement `t[0] = 0` raises an error.

```
t = (1, 2, 3)
t[0] = 0      # TypeError: tuples do not support item assignment
```

```
TypeError: 'tuple' object does not support item assignment
---------------------------------------------------------------------------
TypeError                                 Traceback (most recent call last)
Cell In[63], line 2
      1 t = (1, 2, 3)
----> 2 t[0] = 0      # TypeError: tuples do not support item assignment
TypeError: 'tuple' object does not support item assignment
```

A tuple can, however, contain mutable objects such as lists, and *those* can still be modified:

```
t = ([1, 2, 3], [3, 2, 1])   # the tuple itself cannot change ...
t[0].append(4)               # ... but the lists inside it can
print(t)
```

```
([1, 2, 3, 4], [3, 2, 1])
```

Why use tuples at all? Because a tuple guarantees that a fixed group of values — such as the coordinates `(x, y)` of a point, or the date `(2024, 9, 23)` — stays as it is. Tuples can also be used as dictionary keys, and functions use them to return several values at once (Lecture 4).

3.4 Packing and unpacking

The statement

```
t = 0, 1, 'word'
```

is an example of **tuple packing**: the values `0`, `1` and `'word'` are packed together into a tuple, exactly as in `t = (0, 1, 'word')`.

The reverse operation is called **sequence unpacking**, and it works for any sequence on the right-hand side. Sequence unpacking requires as many variables on the left of the equals sign as there are elements in the sequence. In

```
t = (v1, v2, ..., vN)
t1, t2, ..., tN = t
```

the values `v1`, `v2`, ..., `vN` are unpacked and assigned to the variables `t1`, `t2`, ..., `tN`, respectively: `t1` takes the value `v1`, `t2` takes `v2`, and so on.

In fact, a **multiple assignment** such as `a, b, c = 0, 1, 2`, which we have used since Lecture 1, is just a combination of tuple packing and sequence unpacking: the right-hand side is packed into a tuple, which is immediately unpacked into the variables on the left-hand side. This also gives a neat way to swap two variables.

```
t = ("element-1", "element-2", "element-3")
t1, t2, t3 = t      # sequence unpacking: assign the elements of t to t1,
 ↪  t2 and t3
print(t1, t2, t3)

# packing, then unpacking: (1, 2) is created, then unpacked into a and b
a, b = 1, 2
print(a, b)

a, b = b, a         # swap the values of a and b in one line
print(a, b)
```

```
element-1 element-2 element-3
1 2
2 1
```

4 Dictionaries

4.1 Creating dictionaries

Dictionaries are sometimes called *associative memories* or *associative arrays* in other programming languages. Unlike sequences such as lists and tuples, which are indexed by a range of numbers, dictionaries are indexed by **keys**. A dictionary is best pictured as a real dictionary: you look up a *word* (the key) to find its *definition* (the value).

- A dictionary is a collection of **key–value pairs**, written between curly braces `{...}`, with a colon between each key and its value:

```
my_dict = {key_1: value_1, key_2: value_2, ..., key_N: value_N}
```

- The keys must be **unique** and of an **immutable** type: strings and numbers can always be keys, and so can tuples that contain only strings, numbers or tuples. Lists can never be keys, because they can be modified in place.
- The values can be of any type.
- A dictionary is mutable: we can add, change and delete key–value pairs.

The rule that keys must be immutable matches common sense: we may change the *definition* of a word in a dictionary, but if the word itself could change, we would make a terrible mess.

A dictionary can also be created with the built-in function `dict()` — strictly speaking, the *constructor* of the class `dict`, a term we will explain in Lecture 5 — from a list of `(key, value)` tuples:

```
tel = {'Iron Man': 4098, 'Hulk': 4139, 'Black Widow': 4012}  # define a
 ↪  dictionary
tel_again = dict([('Iron Man', 4098), ('Hulk', 4139), ('Black Widow',
 ↪  4012)])
print(tel)
print(tel == tel_again)
```

```
{'Iron Man': 4098, 'Hulk': 4139, 'Black Widow': 4012}
True
```

4.2 Accessing, adding and deleting elements

- To access a value, use its key in square brackets: `tel['Hulk']`.
- Assigning to a new key adds a new pair; assigning to an existing key replaces its old value, which is forgotten.
- `del tel['Hulk']` deletes a key–value pair.

- `list(d)` returns a list of all the keys, in insertion order; use `sorted(d)` to sort them.
- To check whether a key is in the dictionary, use the keyword `in`.

```
tel = {'Iron Man': 4098, 'Hulk': 4139, 'Black Widow': 4012}
tel['Saitama'] = 5014           # add a key-value pair
tel['Mononoke'] = 6017          # add another one
print("tel =", tel)
print("tel['Iron Man'] =", tel['Iron Man'])   # access an element

del tel['Hulk']                 # delete an element
print("tel =", tel)

print("keys:", list(tel))       # a list of the keys
print("sorted keys:", sorted(tel))
print('Saitama' in tel, 'Hulk' in tel)
```

```
tel = {'Iron Man': 4098, 'Hulk': 4139, 'Black Widow': 4012, 'Saitama': 5014,
↪  'Mononoke': 6017}
tel['Iron Man'] = 4098
tel = {'Iron Man': 4098, 'Black Widow': 4012, 'Saitama': 5014, 'Mononoke': 6017}
keys: ['Iron Man', 'Black Widow', 'Saitama', 'Mononoke']
sorted keys: ['Black Widow', 'Iron Man', 'Mononoke', 'Saitama']
True False
```

Looking up a key that does not exist is an error:

```
tel['Thor']    # KeyError: there is no such key
```

```
KeyError: 'Thor'
---------------------------------------------------------------------------
KeyError                                  Traceback (most recent call last)
Cell In[68], line 1
----> 1 tel['Thor']    # KeyError: there is no such key
KeyError: 'Thor'
```

To avoid the error, check with `in` first, or use the method `get()`, which returns a default value when the key is missing:

```
print(tel.get('Thor', 'unknown'))
print(tel.get('Saitama', 'unknown'))
```

```
unknown
5014
```

4.3 Looping over a dictionary

The methods `keys()`, `values()` and `items()` return, as their names suggest, all the keys, all the values and all the `(key, value)` pairs of a dictionary.

```
tel = {'Iron Man': 4098, 'Hulk': 4139, 'Black Widow': 4012}

print("keys:")
for k in tel.keys():
    print(k, end=' -- ')
print("\n--------------")
print("values:")
for v in tel.values():
    print(v, end=' -- ')
print("\n--------------")
print("key-value pairs:")
for k, v in tel.items():     # k takes the key, v the corresponding value
    print("Telephone of {0:12}: {1}".format(k, v))
```

```
keys:
Iron Man -- Hulk -- Black Widow --
--------------
values:
4098 -- 4139 -- 4012 --
--------------
key-value pairs:
Telephone of Iron Man    : 4098
Telephone of Hulk        : 4139
Telephone of Black Widow : 4012
```

5 Sets

A **set** is an unordered collection with **no duplicate elements**. Sets are written with curly braces, like dictionaries but without colons. They are ideal for removing duplicates and for fast membership tests, and they support the mathematical operations of union, intersection and difference.

```
primes = {2, 3, 5, 7, 7, 3}     # duplicates are removed automatically
evens = {2, 4, 6, 8}
print(primes)
print(5 in primes)              # a fast membership test
print(primes & evens)           # intersection
print(primes | evens)           # union
```

```
print(primes - evens)           # difference
print(set([1, 1, 2, 3, 3, 3]))  # remove the duplicates from a list
empty_set = set()               # NOT {}, which creates an empty
 ↪  dictionary!
```

```
{2, 3, 5, 7}
True
{2}
{2, 3, 4, 5, 6, 7, 8}
{3, 5, 7}
{1, 2, 3}
```

6 Useful looping techniques

When looping over a sequence, the built-in function `enumerate()` gives us the position index and the corresponding value at the same time:

```
for i, v in enumerate(['tic', 'tac', 'toe']):
    print(i, v)

# enumerate() returns an "enumerate" object, which produces the pairs one
 ↪  by one;
# list() collects them so that we can see them.
print(list(enumerate(['tic', 'tac', 'toe'])))
```

```
0 tic
1 tac
2 toe
[(0, 'tic'), (1, 'tac'), (2, 'toe')]
```

To loop over two or more sequences at the same time, pair up their elements with `zip()`:

```
materials = ["steel", "aluminium", "copper"]
densities = [7850, 2700, 8960]                # in kg/m^3
for name, rho in zip(materials, densities):
    print(f"{name:<10} {rho:>5} kg/m^3")
```

```
steel       7850 kg/m^3
aluminium   2700 kg/m^3
copper      8960 kg/m^3
```

Summary

- **Lists** `[...]` are mutable sequences: index them from `0` (or from `-1` at the end), slice them with `[start:stop:step]` (with `stop` excluded), and modify them with assignments and methods such as `append()`, `insert()`, `pop()` and `sort()`.
- Assignment never copies a list; it creates an **alias**. Use `copy()` or `[:]` for a shallow copy and `copy.deepcopy()` for nested lists.
- **Tuples** `(...)` are immutable sequences. Packing and unpacking make multiple assignment and swapping easy.
- **Dictionaries** `{key: value}` map unique, immutable keys to values. Use `in` or `get()` to avoid a `KeyError`, and `items()` to loop over key–value pairs.
- **Sets** `{...}` hold unique elements and support union, intersection and difference.
- `enumerate()` and `zip()` make loops over sequences cleaner.

Exercises

Exercise 1: List methods

Create a list of strings of your choice, for example

```
my_list = ['I', 'am', 'a']
```

Look up the following list methods in the official documentation (or with `help()`), and use each of them at least once on your list:

1. `list.append(x)`
2. `list.extend(iterable)`
3. `list.insert(i, x)`
4. `list.remove(x)`
5. `list.pop()`

After each call, print the whole list to see how it has changed.

Exercise 2: Printing a 2D list in style

A one-dimensional list can be created with

```
list_1d = list(range(5))
```

and a two-dimensional list — a matrix — by nesting several lists inside a list:

```
list_2d = [list(range(5)), list(range(5, 10, 1)), list(range(10, 15, 1))]
```

Create a 1D list and a 2D list of your own. Then print the 1D list on one line, and the 2D list in style, one row per line, for example:

```
[0, 1, 2, 3, 4]

[ 0,  1,  2,  3,  4
  5,  6,  7,  8,  9
 10, 11, 12, 13, 14]
```

Note the alignment of the numbers and commas in the 2D list (it does not have to be perfect).

> **Hint**
>
> As in Exercise 4 of Lecture 2, you need string formatting. This time, align the numbers to the right in a field of suitable width, e.g. `print("{0:>3},".format(value), end="")`. Watch out for the last number of each row, which has no comma after it.

Exercise 3: A dictionary of material densities

Engineers constantly look up material properties. Store the densities of some common materials in a dictionary, with the names as keys and the densities (in kg/m³) as values: steel 7850, aluminium 2700, copper 8960, titanium 4500 and concrete 2400.

1. Print a neatly aligned table of the materials and their densities.
2. For each material, compute the mass of a rectangular plate measuring 0.50 m × 0.20 m × 0.01 m, and print it with two decimal places.
3. Find the lightest and the heaviest material by looping over the dictionary and keeping track of the smallest and largest densities, as in Exercise 5 of Lecture 2.
4. Add glass (2500 kg/m³), remove concrete, and print the names of the materials in alphabetical order.

Exercise 4: Lists versus NumPy for matrix multiplication

A list is a built-in data structure that can hold any number of objects of different types. When it comes to numerical computations, however, lists are slow and inconvenient. The library NumPy, on the other hand, is designed to store large arrays of numbers and to perform algebraic computations with them efficiently, such as the dot product of two vectors, the product of a matrix and a vector, products of higher-order tensors, and so on. In this exercise, we compare a single NumPy function call with the two (or three) nested `for` loops needed to multiply two matrices stored as lists.

Let **A** be a matrix of size $m_1 \times m_2$ and **B** a matrix of size $m_2 \times m_3$. Their product $\mathbf{C} = \mathbf{AB}$ is the $m_1 \times m_3$ matrix defined by

$$C_{ij} = \sum_{k=1}^{m_2} A_{ik} B_{kj} \qquad \text{for all } i = 1, \ldots, m_1 \text{ and } j = 1, \ldots, m_3.$$

With NumPy, computing **C** takes a single line, as the code below shows. Your task is to compute **C** as a list of lists, using `for` loops. Start from the end of the following code.

```
import numpy as np
# m1 = 5, m2 = 3
A = np.array([[1, 2, 3], [4, 5, 6], [7, 8, 9], [10, 11, 12], [13, 14, 15]])
B = np.array([[2, 4], [3, 6], [4, 8]])  # m2 = 3, m3 = 2
# matrix multiplication; A.dot(B) does the same
C = A @ B
print(C)

A = [[1, 2, 3], [4, 5, 6], [7, 8, 9], [10, 11, 12], [13, 14, 15]]
B = [[2, 4], [3, 6], [4, 8]]
m1, m2, m3 = 5, 3, 2
# Write your code here to compute C as a list of lists.
```

```
[[ 20  40]
 [ 47  94]
 [ 74 148]
 [101 202]
 [128 256]]
```

Further reading

- The official Python tutorial: Data Structures.
- Shallow and deep copy operations.